Introduction to Vector

What is vector?

- May think a vector as an intelligent array, that is, vector is a template class that wrap array data and operations on those data.

What is vector? II

- A vector combines the best of a string and an array.
  - Like a string and an array, it uses index.
  - Like a string, it has size method to find out the number of elements.
  - Unlike an array, vector can grow dynamically.
  - In printing an array of integers/double numbers/strings/chars, the code are very similar. Similar for other operations on an array.

What is vector? III

- A vector uses template, a vector of ints, a vector of double, a vector or strings, and so on.
  - vector<int> is a vector of integers
  - vector<double> is a vector of double type numbers.
- In short, vector is easier to use but consume more memory than array. It can do things that an array can do.
  - In some sites like leetcode, vector is counted as array in C++ while array is used in C.

Print a vector vs. print an array

```
#include <iostream>
#include <string>
#include <vector> //vector
using namespace std;
void print(int* arr, int size);
void print(vector<int> vec);

int main() {
    int arr[] = {1, 2, 3, 4};
    int size = sizeof(arr) / sizeof(arr[0]);
    print(arr, size);

    vector<int> vec = {1, 2, 3, 4}; //need compile with c++11 as follows
    //g++ -std=c++11 fileName
    print(vec);

    return 0;
}
```

Talking: Tong Yi

Print a vector vs print an array

```
20  void print(int* arr, int size) {
21      for (int i = 0; i < size; i++)
22          cout << arr[i] << " ";
23
24      cout << endl;
25  }
26
27  void print(vector<int> vec) {
28      for (int i = 0; i < vec.size(); i++)
29          cout << vec[i] << " ";
30
31      cout << endl;
32  }
```

Why do we use vectors over arrays?

- Link
- Vectors are sequence containers representing arrays that can change in size.
- Just **like arrays**, vectors use contiguous storage locations for their elements, which means that their elements can also be accessed using offsets on regular pointers to its elements, and just as efficiently as in arrays.
- But **unlike arrays**, their size can change dynamically, with their storage being handled automatically by the container.

Why vector?

- Internally, vectors use a **dynamically allocated array** to store their elements. This array may need to be reallocated in order to grow in size when new elements are inserted, which implies allocating a new array and moving all elements to it. This is a relatively expensive task in terms of processing time, and thus,
- vectors do not reallocate each time an element is added to the container.
- Therefore, compared to arrays, vectors consume more memory in exchange for the ability to manage storage and grow dynamically in an efficient way.